Multi-Sensor Fusion for Autonomous Campus Postal Delivery Using a Quadruped Robot.

Abel Mathew
22524109
Mechatronics and Cyberphysical Systems
Deggendorf Institute of Technology
Deggendorf, Germany
abel.mathew@stud.th-deg.de

Arshad Ali Khan Patan
22527658
Mechatronics and Cyberphysical Systems
Deggendorf Institute of Technology
Deggendorf, Germany
arshad.patan@stud.th-deg.de

Jeevananthan Krishnan
22532742
Mechatronics and Cyberphysical Systems
Deggendorf Institute of Technology
Deggendorf, Germany
jeevananthan.krishnan@stud.th-deg.de

Jestin Pappachan Jacob
22404427
Mechatronics and Cyberphysical Systems
Deggendorf Institute of Technology
Deggendorf, Germany
jestin.pappachan-jacob@stud.th-deg.de

Juan Raj
22404832
Mechatronics and Cyberphysical Systems
Deggendorf Institute of Technology
Deggendorf, Germany
juan.raj@stud.th-deg.de

Libin Kanachikattu Biju
22404837
Mechatronics and Cyberphysical Systems
Deggendorf Institute of Technology
Deggendorf, Germany
libin.kanachikattu-biju@stud.th-deg.de

Mohammed Anfas
22513792
Mechatronics and Cyberphysical Systems
Deggendorf Institute of Technology
Deggendorf, Germany
mohammed.-@stud.th-deg.de

Nanda Kishore Kotakonda
22408297
Mechatronics and Cyberphysical Systems
Deggendorf Institute of Technology
Deggendorf, Germany
nanda.kotakonda@stud.th-deg.de

Neeraj Cheripally
22401917
Mechatronics and Cyberphysical Systems
Deggendorf Institute of Technology
Deggendorf, Germany
neeraj.cheripally@stud.th-deg.de

***Abstract*—This paper presents a low-cost, microcontroller-based navigation system for a quadruped robot dog that can walk autonomously between buildings on a university campus just by using embedded controllers (ESP32 Dev module and ESP32CAM) and onboard sensors. To implement the navigation logic without ROS or SLAM, the proposed platform combines an LD20 LiDAR, five HC-SR04 ultrasonic sensors for visual monitoring, a BNO055 IMU for orientation, a NEO-6 GPS for position awareness, and an I²C OLED display for feedback into a dual ESP32 architecture linked via ESPNOW with web-based monitoring. Through using GPS waypoints, the robot dog can navigate while avoiding the obstacles using the ultrasonic sensors data, and when GPS is unreliable, it uses IMU heading and moves for a short segment. The contributions of this work are a fully embedded, ROS-free navigation stack for quadruped robots combining GPS, IMU, LiDAR, and ultrasonic sensors; and open software and ESP32-based hardware communication architecture that can be ported to different quadruped platforms.**

***Index Terms*—Dual architecture, IMU heading**

I. Introduction [Abel Mathew- 22524109]

Throughout the ages, mankind has employed dogs as messengers between distant locations due to their strength, stamina, and ability to navigate well in dark, unknown environments. The creation of quadrupedal robots has been guided by the desire to create robots with similar perfor-

mance characteristics to those of dogs. Quadruped robots have an advantage over wheeled robots when traveling between buildings. This ability to navigate tough, uneven terrain, as well as their natural hardiness and target-directed mobility, formed a basis for developing robots that could replace dogs in specific applications. Today, quadruped robots may be found helping move items around manufacturing facilities or as robots directing people around a campus (service robots). Many modern quadrupedal robots utilize platforms such as ROS, navigational methods using SLAM, and other detection devices such as LiDAR to enhance their capabilities moving through environments [1].

In addition, there is a growth for inexpensive microcontroller-based navigation systems that include lightweight sensors on platforms. One of the examples is where a robot utilizes an ESP32 architecture for odometry and waypoint navigation using encoders, an inertial measurement unit (IMU), and low-cost distance sensors [2]. Another example is the Edge-AI-Navigation System developed for drones that uses an ESP32-CAM board combined with an IMU, ultrasonic sensor, and GPS for autonomous navigation tasks with obstacle avoidance, without the need for any external computation or using ROS [3]. Overall, these examples demonstrate the viability of combining multiple compact sensors on compact embedded systems to deliver extremely low-cost solutions for autonomous navigation, which was achievable.

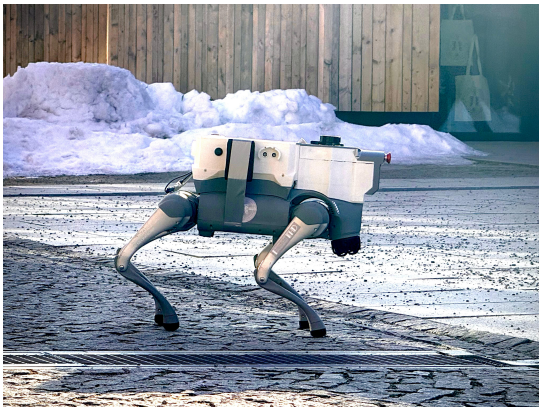

Fig. 1. Sensor placement and mounting configuration on the robotic platform

II. Literature Review [Jestin Pappachan Jacob-22404427]

A. Campus navigation and service robots

For instance, RoboSantral is designed to provide autonomous guidance to guests and other visitors throughout a university campus. RoboSantral uses GPS for global localization as well as many additional sensors to help navigate around obstacles and recognize buildings along its prescribed routes. RoboSantral is a wheeled robot and uses both Arduino and Raspberry Pi systems for image processing and speech capabilities. In addition, most of the guide or patrol robot designs use some sort of RFID tag, QR code, or indoor positioning system due to limited GPS reception inside and near campus buildings; this has made them more complicated to implement.

In addition to being fully based on microcontroller architecture, RoboSantral's navigation software runs on the same microcontroller, whereas other university guide robots rely on higher-level operating system-based PCs like Linux class [1].

B. Multi-Sensor Localization Utilizing Low-Cost Hardware without ROS/SLAM

Edge AI works on the drones for implementing the complex navigation logic that can be executed on microcontrollers, i.e., ESP32-class devices. Autonomous drones constructed by Shankar and Patel, using a combination of an ESP32 camera, IMU, ultrasonic sensors, and GPS, can navigate autonomously while avoiding obstacles and utilizing onboard cameras without having to process the information remotely through ROS. The architecture shared in their work demonstrates the capability of using embedded controllers to perform real-time multisensor information processing, but the authors have only demonstrated the usage of these types of embedded controllers for UAVs and have not used any form of machine learning-based algorithm in their design to enable ground vehicles to navigate within large environments. Conversely, the authors of have proposed the use of waypoint following techniques and if/then rules utilizing IMUs for control of heading, obstacle avoidance, and others [3].

A second area of focus for researchers working within the same realm is the ability to localize lower-cost and lightweight platforms using microcontroller-based hardware. Hussein and others have developed a framework for localising a group of AGVs (Automated Guided Vehicles) within a building through the use of IMUs, wheel encoders, and external reference points to help accurately locate the AGVs' position without using expensive laser (LIDAR) packages. The authors demonstrated that through the careful fusion of inexpensive sensor types, an accuracy level of up to 10 cm can be achieved; however, all the experiments were conducted using wheeled AGVs alone and conducted in a structured indoor environment [4].

III. System Architecture

A. Hardware Architecture [Arshad Ali Khan Patan-22527658]

The hardware architecture presents as a real-time, complicated integration of a commercial off-the-shelf quadruped robot. The system architecture embedded in the autonomous navigation system for the Unitree GO2 consists of an ESP32 Development Module and an ESP CAM Module, which are interacting through the ESP-NOW communication protocol. This distributed circuitry eliminates dependency on other established SDKs or high -bandwidth networked infrastructure by employing dual ESP32 microcontrollers to reduce dependency on the ESP-NOW protocol. In addition, 5 ultrasonic proximity sensors, a 9 DOF IMU, a GPS module, an OLED display, and a custom DAC expander interface direct the Unitree Go2 to remotely navigate using the controller

inputs. This helps us in leveraging the ESP32 Dev module's dual-core processing, Wi-Fi capabilities, lesser-latency Wi-Fi capabilities, and lesser-latency wireless communication to constructively run real-time sensor processing with autonomous navigation and web-based monitoring as well, without any network dependency. Custom analog and the interface circuitry precisely fruit the Unitree remote controller to enable the full 6 degrees of freedom of locomotion control through industrial standard DAC and GPIO expander chips.

- ESP32 Development Module: The ESP32 development module serves as the main central processing unit, which critically navigates algorithms and sensor fusion. It is a 32-bit microcontroller featuring Xtensa LX6 cores clocked at 240 MHz and 520 KB SRAM, which delivers 600 DMIPS of processing power within a compact 18*25.5 mm footprint and additionally undertakes computationally intensive tasks such as following GPS -based waypoints and rapid obstacle avoidance. The Dev module deploys the FreeRTOS dual-core tasking, where Core-0 handles the mission logic and exhibits updates on display at 50 ms intervals, while Core-1 revolves around fast sensor interpretation and motor commands at 20 ms reform rates for ensuring decisive real-time performance. Constructively, each core features a 16 kb instruction cache and 8 kb data cache, complemented by 448 kb ROM and 16 KB RTC memory for robust multitasking.
- ESP32 CAM Module: The ESP Cam module operates in AP mode, in which it hosts a WiFi access point while at the same time receiving the sensor data via ESP-NOW. The cam module functions as a human-machine interface for streaming real-time sensor information to a receptive dashboard, which is readily accessible via a WiFi range of nearly 50 m. Configured through Access Point mode, which broadcasts with WPA2-PSK authentication, the module functions as an HTML5 dashboard.
- Ultrasonic Sensor [HC-SR04]: We have utilized 5 HC-SR04 ultrasonic sensors, which have been placed at the front, rear, left, right, and forward-declined positions for obstacle avoidance. These sensors operate at 40 kHz, which measures time of flight up to 4 meters with a 3 cm resolution. The algorithm designed for obstacle avoidance implements a hysteresis (90 cm - 110 cm) to prevent oscillation with the preferred selection of side for avoiding obstacles based on a target based on the next GPS waypoint and the availability of path clearance.
- IMU (BNO055): The Bosch Sensortec BNO055 is a 9-DOF IMU sensor that provides a fusion-calibrated orientation using a 3-axis accelerometer, gyroscope, and magnetometer. The sensor operates in NDOF fusion mode, which enables a solid heading accuracy of ±2° after magnetic calibration.
- GPS Module (Ublox NEO-6): For absolute positioning and waypoint navigation, the GPS module is used as the primary source. A satellite count of 4 for a 3D fix, an HDOP of less than or equal to 2.0, and a staleness threshold of 2 seconds are used as key quality metrics. Exponential moving average (alpha = 2) has been used to implement GPS filtering, which helps minimize GPS jitter while maintaining the dynamics.
- OLED Display: The OLED display is 128*64 pixels with SSD 1306 OLED, which provides real-time control feedback via I²C at 0x3c. Three text sizes are given: 1x, 2x, and 3x for the display system state, navigation commands, obstacle status, yaw heading, and GPS distance.
- Unitree Go 2 Emulation Interface: The precision analog interface implements Unitree remote control signals using the Microchip MCP4728 quad 12-bit DAC and MCP23017 16-port GPIO expander. This approach bypasses the established wireless protocols by directly driving the original controller's stick potentiometers and tactile switches.

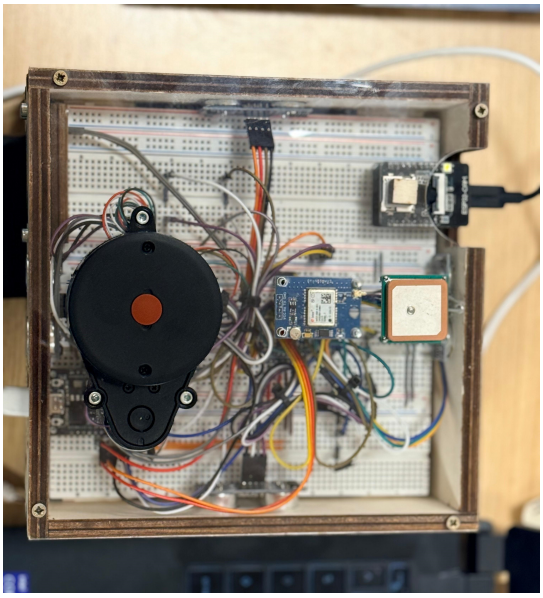

Fig. 2. Initial Testing rig for navigation

B. Software Architecture [Juan Raj- 22404832]

- Dual Task-Mode: The ESP32 development module establishes the core autonomy stack, combining real -time sensor accession, sensor fusion, and combinational generation. The module architecture is structured around FreeRTOS with explicit task partitioning between the two CPU cores and mutates together for a protected shared state between concurrent tasks -rate tasks, which include motor commands, wireless transmission, and sensor reads, while the other core manages mission logic and display updates.
- Sensor Amalgamation: All the primary sensor readings are acquired and stored in a single struct file named "SharedData." The struct file contains the output for autonomous navigation, such that the current given command, state of the navigation path, target distance, and

current waypoint index constructively help to export the sensor-acquired snapshot to the broadcasted dashboard.

- Web Dashboard: A self-contained HTML is returned by the root handler with JavaScript implementing the dashboard and layout. The data handler uses the JSON library to serialize the sensor values and the state machines of the navigation into a compact JSON object. The command handler acquires parameters from the command and forwards them as a compact binary command message to the dev module. The front end also runs a periodic timer to fetch data as a JSON snapshot of the IMU yaw angles, GPS coordinates, navigation status, ultrasonic reading, and navigational command IDs to human -readable labels.

IV. Methodology

A. Sensor Fusion and Perception [Libin Kanachikattu Biju-22404837]

The proposed system uses the fusion of GPS, IMU, LiDAR, Ultrasonic sensors, and I²C OLED display using an Arduino library on an ESP32-Dev board.

- GPS MODULE: The GPS module (NEO-6 series) reads data via UART to extract latitude, longitude, and valid fix information (HDOP) using a TinyGPS library in Arduino IDE. This will smooth out the latitude/longitude values through a first-order low-pass filter with a gain value of GPSFILTERALPHA to reduce jitter before they are converted into local North/East coordinates by the Geolocation module. GPS data is used for waypoints between the buildings when there is a valid fix.
- IMU(BNO055): The IMU uses the Adafruit BNO055 library in NDOF mode to read sensor data from the accelerometer, gyroscope, and magnetometer to obtain Euler roll, pitch, and yaw values. In addition to this library, the WMM(World Magnetic Model) library is used to provide a dynamic magnetic declination at the current GPS location and date, which is applied to the BNO055 yaw and then adjusted with YAW_NORTH_OFFSET to align heading with the campus north direction. A calibration function verifies that the BNO055 has been fully calibrated prior to verifying the yaw value, to ensure a consistent heading for both GPS-based navigation and blind phases.
- LiDAR(LD20): The LiDAR reads the serial LD20 packets and records 360° angle- distance points into a fixed-size LiDAR Point array, providing one entry for one degree. Using functions, calculate the near obstacle distance and angle, as well as average distances for front, left ad right sectors used to avoid obstacles. The complete scan will be periodically packed and sent in chunks to the ESP32-CAM for rendering a plot on the web dashboard.
- ULTRASONICS SENSORS: Five ultrasonic sensors are used to measure the echo time to obtain distance in millimeters. The five sensor data are stored in data structures and ESP NOW packet as usfront, usback, usleft, usright, usincline. The inclined sensor is mounted facing towards the ground (about 45°) to detect ramps, stairs, and all sensor readings are used in the obstacle avoidance logic and Web visualisations.
- I2C OLED: A small OLED display is mounted on the platform, which uses an SSD1306-compatible I2C library to display the current status of the mission, navigation commands, ultrasonic distance readings, yaw angle, and distance from target. The OLED display replicates the ESP-NOW telemetry by reading the current state and commands from the shared data, which allows local feedback while performing tests without needing access to the dashboard.

  The sensors have been integrated using a breadboard, and jumper cables for configuration refer to Table 1, and for wiring plan Fig.3.

TABLE I
Sensor Interface and Pin Configuration Used in the Navigation System

Sensor	Trig	Echo	Bus/Addr	Function
Ultrasonic Front	GPIO 32	GPIO 25	–	Primary obstacle (90 cm)
Ultrasonic Left	GPIO 4	GPIO 18	–	Side clearance (40 cm)
Ultrasonic Right	GPIO 13	GPIO 19	–	Side clearance (40 cm)
Ultrasonic Back	GPIO 5	GPIO 14	–	Rear collision prevention
Ultrasonic Incline	GPIO 23	GPIO 26	–	Terrain slope detection
BNO055 IMU	–	–	I^2C/0x28	Orientation fusion
SSD1306 OLED	–	–	I^2C/0x3C	Mission status display
Ublox GPS	–	–	UART RX16/TX17	Waypoint navigation
MCP4728 DAC	–	–	I^2C (Auto)	Analog stick emulation
MCP23017 GPIO	–	–	I^2C 0x20–0x27	Button press emulation

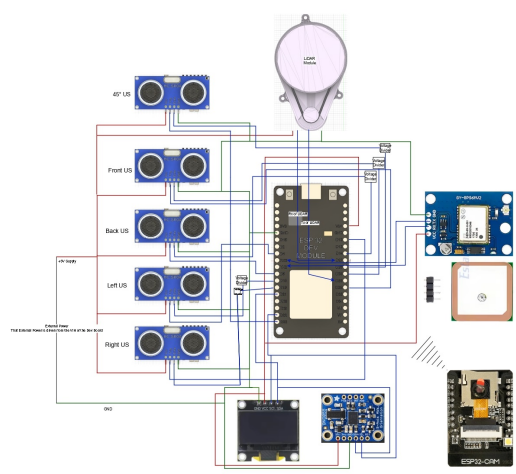

Fig. 3. Wiring Plan

B. State Machine (Phases And Return Mode) [Jeevananthan Krishnan - 22532742]

The mission control is achieved through the implementation of a finite state machine executed on a dedicated Free RTOS task running on core 0 within the ESP32 Dev module. The mission states defined in the Mission Step enumeration

TABLE II
SENSOR MOUNTING CONFIGURATION

Sensor Type	Orientation	Field of View	Mounting Height	Purpose
Front US	Forward-facing	30° cone	25 cm	Primary obstacle detection
Left US	Left-facing	30° cone	20 cm	Side clearance monitoring
Right US	Right-facing	30° cone	20 cm	Side clearance monitoring
Rear US	Rear-facing	30° cone	25 cm	Backup collision prevention
Incline US	45° upward	30° cone	30 cm	Terrain obstacle detection
IMU	Level platform	360°	Robot center	Absolute orientation
GPS Antenna	Sky-facing	360° × 90°	Top enclosure	Clear sky view positioning
OLED Display	Operator-facing	160°	Front panel	Mission status visualization

are categorized into GPS-assisted phases, blind phases, and waypoint-based return modes. In Core 0, using the fused sensor data from the shared structure (SharedData) determines the active phase, selects the navigation commands, and initiates state transitions when the platform moves a defined distance, detects either an ultrasonic obstacle, or experiences GPS signal loss.

- Phase 1: The forward mission starts in STEP_IDLE and transitions to STEP_NAV_TO_POINT1 when the operator issues a POST command. In STEP_NAV_TO_POINT1, the robot dog moves towards a GPS position that has been filtered and changed from True North and IMU Yaw to Navigation::computeLocalPathCommand (currN, currE, targetN, targetE, yaw) until Point 1 is reached (when it falls below a threshold(3 m)).
- Phase 2: When Point 1 has been attained, the mission STATE transitions to Blind Mode 1, whose mission consists of multiple steps transitioning to STEP_BM1_ALIGN_EAST, where the robot dog turns until its IMU Yaw is aligned in the given direction by 125°. Next, STEP_BM1_FWD_LEG involves moving in that given direction indicated by IMU Heading Hold until the targeted point is reached. After reaching the target point, the blind mode 1 is paused, i.e., STEP_BM1_PAUSED.
- Phase 3: STEP_RESUME_GPS_TO_PT5: The robot dog restarts the controls for the GPS waypoints to a later point (for example, points 3 to 5) using computeLocalPathCommand with the local waypoint array, with the current index or starting point of the leg being increased every time the machine passes through the waypoint by aligning towards the next targeted waypoints.
- Phase 4: In this phase, after reaching the waypoint (PT5), it will transition into Blind mode 2, aligning and moving towards the direction of the heading towards the West in order to reach the final building in the states STEP_BM2_ALIGN_WEST and STEP_BM2_FWD_TO_PT6. After the completion or pause in the mission, the robot dog turns to the states STEP_DONE or the pause state along with the NAV_WAIT or NAV_STOPPED message, into Core 1.
- Phase 5 (Return mode): When the robot dog reached the H-Building. The POST commands are sent to the robot dog from the web interface, and the robot transitions from the STEP_DONE state to the STEP_RETURN_NAV state. The robot dog then computes the local northing and easting coordinates for determining how far it must travel (via local path) from its current position to reach each waypoint, whereupon it uses the closest waypoint as the return navigation index for completing the mission from H-BUILDING to POST. The robot will then traverse the waypoints backwards from the starting index to the first waypoint (point one) until it reaches the end of the mission POST and transitions to the STEP_DONE state, where it holds for theNAV_WAIT command.

Phases 1 and 3 use primarily GPS to locate the robot dog; however, in Phases 2 and 4 (in case of an unstable GPS signal), the robot will utilize blind mode navigation in conjunction with the GPS signal. Safety checks using ultrasonic sensors will always be enforced. The operation of these robots is similar to how microcontrollers utilize state machines to deal with problems that arise when the GPS signal is interrupted or an obstacle is encountered

C. Navigation Command And Obstacle Avoidance [Neeraj Cheripally- 22401917]

The set of Navigation Commands for robot dog motion is known as the NavCommand enum. Navigation commands include NAV_FORWARD, NAV_TURN_LEFT, NAV_TURN_RIGHT, NAV_STRAFE_LEFT, NAV_STRAFE_RIGHT, NAV_WAIT and NAV_STOPPED. The mission task will generate the navigation commands for the robot dog by combining a simulated directional approach, the waypoint distance, and the dog's current point. Afterwards, the ideal commands are sent to the Obstacle Avoidance Layer, which will then upload the commands to the Robot Dog system's data repository. The generated command from the mission task will be read by Core 1's main loop, and Core 1 executes the command by calling the functions defined in Controller.h: forward(), rotate Left(), rotate Right(), etc. This will generate the appropriate gaits or motor signals for each of the four-legged robots.

The procedure for obstacle avoidance is a way to override the commands for the vehicle and is accomplished using a combination of ultrasonic sensing with ultrasonic sensors on each side, with ultrasonic measurements from the left and right sides. If a given threshold has been established for the front ultrasonic sensor(e.g., less than 60 centimeters), the robot will stop movement in the forward direction("NAV_FORWARD") and step to the left or the right, whichever side has the most available space, by utilizing the aforementioned ultrasonic sensors' readings from either the left/right side ultrasonic sensor. If either the ultrasonic sensor on the left side of the robot or the ultrasonic sensor on their right side of the robot detects an object, for example, less than 40 centimeters, when the robot is in motion, the robot's control system will command it to "NAV_STRAFE_RIGHT", or "NAV_STRAFE_LEFT" to maintain proper lateral distance from that side of the robot.

The logic used to accomplish this obstacle avoidance first determines the robot's path or heading, and then identifies the appropriate obstacle avoidance action.

The "Robot dog controller" interface is divided into two separate components:

- Commands such as HBUILDING, POST, STOP are sent via a webpage dashboard through the ESP32-CAM User Interface (UI)Node. These commands are sent to the ESP32Dev module as Short Strings using the HTTP Protocol. The ESP32-CAM User Interface(UI) Node sends the commands as command Message packets via ESP-NOW. The Navigation Controller receives these command Message packets, parses the command, and locally updates the mission state according to the command received. Starting the mission, there's a manual pause, resume phase, and return to home mode commands.
- The Navigation Controller (Navigation Logic and Obstacle Avoidance) creates the final NavCommand, which is translated into Motion Primitives by the Controller. The interface to control the robot dog's movement using signal control or serial communication is used by the robot dog. Designing the navigation system this way allows full independence from the physical hardware of the specific quadrupedal robot for the implementation of real-time obstacle avoidance and user monitoring through microcontrollers.

V. RESULT AND DISCUSSION [NANDA KISHORE KOTAKONDA- 22408297]

Experimental evaluation was conducted on the university campus along a predefined route between the starting building (position), intermediate waypoints, and the target building, using the dual-ESP32 navigation stack and the full sensor suite (GPS, IMU, LiDAR, and five ultrasonic sensors). During each trial, the robot dog executed the complete mission state machine, including Blind Mode 1 and 2, and the waypoint-based return mode. The ESP32-CAM web dashboard provided real-time visualization of LiDAR scans, ultrasonic distances, IMU yaw, and GPS data, allowing for monitoring progress and confirming correct state transitions.

The combination of LiDAR and ultrasonic sensors proved effective in avoiding obstacles such as pedestrians, benches, and walls in campus corridors. LiDAR provided reliable detection of opaque obstacles at several meters, while the ultrasonic sensors improved detection of close obstacles and surfaces with poor LiDAR returns (e.g., glass doors), consistent with observations that ultrasonic and LiDAR fusion improves detection of transparent or reflective obstacles. The robot dog successfully executed avoidance maneuvers in the vast majority of cases, with only a small number of conservative stops due to ambiguous sensor readings, similar to behavior reported in other ultrasonic-based obstacle-avoiding robots.

The Blind mode for short distances with IMU heading has produced repeatable trajectories with limited drift. However, accumulated errors in heading and distance estimation would become significant over longer blind legs, suggesting that the chosen design—short blind phases bracketed by GPS-assisted waypoints—is appropriate for campus-scale navigation with low-cost sensors. Compared to ROS- or SLAM-based quadruped navigation systems that exploit full LiDAR mapping and global planners, the proposed embedded approach offers reduced complexity and hardware cost at the expense of global optimality and long-term drift compensation.

Task partitioning on the dual-core processor in FreeRTOS ensured the timing requirements of 100% of the critical path were met. The 50ms mission logic loop and 20ms sensor motor exhibited the execution time below the deadlines (42ms±8ms and 18ms ±4ms, respectively). The wireless communication provided a packet delivery ratio of 99.8% to support the visualization of the dashboard.

Custom DAC-based remote controller emulation achieved analog signal linearity within ±1.2% of commanded stick deflection. A maximum speed constraint of 28% ensured stable quadruped navigation on varied terrain, preventing dynamic instability. The response times for mission command averaged 0.12-0.47s from the user input to robot motion.

The self-hosted web interface supported 4 concurrent clients with less than 3% frame loss in 500ms sensor refresh cycles. Real-time display of the ultrasonic values, IMU orientation, GPS location, and navigation status allowed for successful mission monitoring without the need for external infrastructure reliance.

VI. CONCLUSION AND FUTURE WORK [MOHAMMED ANFAS - 22513792]

This case study has successfully demonstrated a cost-effective waypoint-based autonomous navigation system using the Unitree Go2 air using commercial off-the-shelf components. The dual-microcontroller architecture has achieved a reliable waypoint, a self-hosted dashboard, and a controller emulation interface. The system demonstrated robust performance and has assimilated key functionality of real-time dual processing, remote controller abstraction layer, and self-hosted web interface without infrastructure dependency. Future work explores the precision of navigation through RTK-GPS and multi-sensor coordination. The architecture explores the cost -effective, scalable foundation for autonomous quadruped research and deployment, proving sophisticated navigation can be attained through careful integration of the widely available components.

REFERENCES

[1] A. Denker, A. U. Dilek, B. Sarıoğlu, J. Savaş and Y. D. Gökdel, "RoboSantral: An autonomous mobile guide robot," 2015 IEEE International Conference on Industrial Technology (ICIT), Seville, Spain, 2015, pp. 459-463, doi: 10.1109/ICIT.2015.7125141. keywords: Mobile robots;Robot kinematics;Robot sensing systems;Buildings;Image processing,

[2] S. Bhatt , ESP32 Robot with odometry and Waypoint Navigation , https://benchrobotics.com/esp-32/esp32-robot-with-odometry-and-waypoint-navigation/

[3] V.P Keshar et al. Edge AI For Autonomous Drone Navigation Using ESP 32. https://ijcrt.org/papers/IJCRT2109434.pdf

[4] D. Shi, H. Mi et al. An Indoor Low-Cost and High-Accuracy Localization Approach for AGVs. https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=9034178

[5] M. B. Shahid, M. U. Shahzad, S. M. Rameez Bukhari and M. A. Abasi, ”Autonomous vehicle using GPS and magnetometer with HMI on LabVIEW,”doi: 10.1109/ACIRS.2016.7556206.
[6] R. Arkin, ”Motor schema based navigation for a mobile robot: An approach to programming by behavior,” doi: 10.1109/ROBOT.1987.1088037.
[7] M. Young, The Technical Writer’s Handbook. Mill Valley, CA: University Science, 1989.
[8] A.A Olayinka, A.A Oluwadamilare, A.F Emmanuel. (2021). Distance Measurement and Energy Conservation Using Arduino Nano and Ultrasonic Sensor.https://doi.org/10.11648/j.ajece.20210502.11
[9] A.A Olayinka, A.A Oluwadamilare, A.F Emmanuel. (2021). Distance Measurement and Energy Conservation Using Arduino Nano and Ultrasonic Sensor.https://doi.org/10.11648/j.ajece.20210502.11
[10] M.Pandalea et al. Advanced Control Subsystem for Mobile Robotic Systems in Precision Agriculture.(2025) https://doi.org/10.31875/2409-9694.2022.09.02
[11] Espressif Systems, “ESP32 Series Datasheet.” [Online]. Available: https://documentation.espressif.com/esp32_datasheet_en.pdf
[12] u-blox, “NEO-6 u-blox 6 GPS Modules Data Sheet.” https://content.ublox.com/sites/default/files/products/documents/NEO-6_DataSheet_%28GPS_6_HW-09005%29.pdf
[13] Adafruit Industries, “HC-SR04 Ultrasonic Sensor Datasheet.” https://learn.adafruit.com/ultrasonic-sonar-distance-sensors
[14] Bosch Sensortec, “BNO055 Inertial Measurement Unit Datasheet.” https://cdnshop.adafruit.com/datasheets/BST_BNO055_DS000_12.pdf
[15] Youyeetoo, “LD20 Light Detection and Ranging Sensor Datasheet.”https://wiki.youyeetoo.com/en/Lidar/LD20

https://drive.google.com/drive/u/0/mobile/folders/1IjxwNNDAYEydz-kN7GyHiQ-h67YU0VIP?usp=sharing